

SKT 알레프 A반 · 7번째 과제

모찌의 일기장 2

6번째 과제의 다이어리에 가입·로그인을 붙여 내 기록을 나만 볼 수 있게 만들고, "막았다"는 문장 대신 막히는 장면(요청과 응답)을 남긴 기록. 그리고 그 잠긴 앱을 5일 동안 쓰면서 3일차 전에 계획 규칙을 하나만 바꿔 본 실험
과제 제출 보고서

제출자	김대훈
제출일	2026년 9월 8일
결과물	chacha1650a.github.io/assignment-repository/7번째 과제(로그인 기능)/index.html
소스	github.com/chacha1650a/assignment-repository
인증 서버	mochi-diary-auth-backend.onrender.com

무엇을 만들었나

6번째 과제에서 만든 **모찌의 일기장**을 그대로 이어받아, 저장소를 "브라우저 / 공용 API 키 서버"에서 "**내 계정 전용 서버**"로 바꿨습니다. 6번은 키 하나를 아는 사람이면 누구나 같은 기록을 읽고 썼지만, 7번은 계정마다 자료가 갈립니다.

카드	요구사항	구현
1	인증 방식 선택과 이유, 가입·로그인·로그아웃	직접 구현 + bcryptjs 3.0.3 . 인증 서비스(Firebase)-JWT·쿠키 세션을 검토하고 고르지 않은 이유를 2절에 기록
2	비밀번호를 되돌릴 수 없게 보관	bcrypt(반복 12회). 같은 비밀번호로 만든 두 계정의 저장값이 서로 다른 것을 DB를 열어 확인(4절)
3	로그인 상태 유지와 로그아웃 시 실제 차단	서버에 저장하는 세션 토큰. DB에는 원문이 아니라 HMAC-SHA256(토큰, SESSION_SECRET) 만 저장. 12시간 만료, 로그아웃·비밀번호 변경 시 서버가 즉시 취소
4	남의 자료 차단(읽기·수정·삭제, 양방향)	모든 조회에 WHERE user_id = ? . 남의 기록 id로 접근하면 403이 아니라 404 . 거절 앞뒤로 건수·내용이 그대로임을 확인(4절)
5	설명서 6항목 + 잠긴 앱으로 5일 사용, 3일차 전 규칙 1개 변경	설명서를 6항목으로 나눠 작성(⑥ 포함). 2026-09-01~05 기록, 09-02 22:10(KST)에 계획 규칙 변경(6절)

6번 과제 백엔드는 한 줄도 고치지 않았습니다. 6번 제출물이 지금도 그대로 열려야 하기 때문에, 7번용 인증 서버는 **mochi-diary-auth-backend/** 라는 별도 폴더·별도 서비스로 새로 만들어 배포했습니다. 이어받은 6번 최종 commit은 **f8d15118** (2026-09-04)입니다.

무엇으로 붙였고, 왜 그걸 골랐나

말긴 일	무엇으로	이름과 버전
비밀번호 되돌릴 수 없게 만들기	라이브러리	bcryptjs 3.0.3 (알고리즘 bcrypt, 반복 12회)
사람을 알아보기	직접 구현	서버 저장 세션 토큰 — <code>node:crypto</code> <code>randomBytes(32)</code> + HMAC-SHA256
웹 서버 / CORS / 환경변수	라이브러리	express 4.22.2 · cors 2.8.6 · dotenv 16.6.1
데이터베이스	Node 내장	<code>node:sqlite</code> (Node 22.13 이상 내장, 네이티브 빌드 불필요)

비밀번호에 bcrypt를 고른 이유

한 문장으로: 비밀번호를 다루는 부분은 직접 만들면 반드시 어딘가 틀리기 때문에, 오래 검증된 표준을 그대로 쓰고 싶었습니다.

- 계정마다 다른 소금값(salt)을 알아서 붙여 줍니다. 제가 salt를 만들고 붙이고 꺼내는 코드를 짜면 그 자리에서 실수가 납니다.
- **일부러 느립니다(반복 12회)**. 훔쳐 간 사람이 빠르게 다 대입해 보는 것을 어렵게 합니다.
- 비교도 `bcrypt.compareSync`에 맡겼습니다. 문자열을 `===`로 비교하면 걸린 시간으로 정답을 알아내는 공격이 가능한데, 라이브러리가 그 부분까지 처리합니다.
- `bcryptjs`는 순수 자바스크립트라 **네이티브 빌드가 필요 없어** 무료 배포 환경에서 빌드가 깨지지 않습니다.

함께 검토했지만 고르지 않은 것

검토한 것	고르지 않은 이유
Firebase Authentication (인증 서비스)	가입·로그인은 편해지지만 "남의 자료가 안 열린다"를 만드는 소스가 내 코드에 남지 않아, 이 과제가 요구하는 "어디를 고쳤나 / 무엇이 거절을 만드나"를 보일 수가 없었음
JWT (<code>jsonwebtoken</code>)	서명만 맞으면 통과시키는 구조라 로그아웃해도 만료 전까지 그 토큰이 계속 통함. 통과 기준(로그아웃 뒤 거절)을 만들려면 결국 서버에 취소 목록이 필요해, 처음부터 서버 세션이 단순했음
express-session + 쿠키	화면은 GitHub Pages, 서버는 Render로 주소가 달라 크로스 사이트 쿠키를 써야 하는데, 요즘 브라우저가 이를 막는 경우가 많아 심사자 환경에서 로그인이 안 될 위험이 컸음
SHA-256 한 번만 걸어 저장	빠른 것이 장점인 해시라 오히려 다 대입해 보기 좋음. 비밀번호 보관에는 일부러 느린 함수를 써야 한다고 판단
argon2 / scrypt / PBKDF2	argon2는 네이티브 빌드가 필요해 무료 배포에서 자주 깨지고, scrypt-PBKDF2는 salt 생성·저장 형식을 직접 설계해야 해서 "직접 만들지 않는다"는 원칙에서 멀어짐

어디를 어떻게 고쳤나

데이터베이스가 달라진 지점 — 주인이 생겼다

6번: `entries(date PRIMARY KEY, mood, text, habits, updated_at)`
 -> 날짜 하나에 기록 하나. 누구 것인지 적는 칸 자체가 없음

7번: `users(id, email, email_lower UNIQUE, password_hash, created_at)`
`sessions(id, user_id, token_hash UNIQUE, created_at, expires_at, revoked_at)`
`entries(id, user_id, date, ..., UNIQUE(user_id, date))`
`settings(user_id PRIMARY KEY, question, metric, unit, calc_rule, plan_rule)`
`rule_changes(id, user_id, changed_at, before_rule, after_rule, reason, based_on)`
 -> `user_id` 는 전부 `users(id)` 를 가리키고 `ON DELETE CASCADE`

네 흐름이 소스의 어디를 지나는가

흐름	지나는 자리
가입	화면 <code>app.js:195</code> (폼 제출) → <code>app.js:29</code> (<code>api()</code>) → <code>server.js:192</code> (형식·8자 검사, 중복 이메일 409) → <code>server.js:111</code> (<code>hashPassword</code>) → <code>server.js:125</code> (<code>issueSession</code>) → <code>app.js:245</code> (<code>enterApp</code>)
로그인	<code>app.js:195</code> → <code>server.js:220</code> → <code>server.js:226~231</code> (계정 없음과 비번 틀림을 한 갈래로 합쳐 같은 응답) → <code>server.js:114</code> (<code>verifyPassword</code>) → <code>server.js:125</code> (토큰·만료 발급)
로그아웃	<code>app.js:296</code> → <code>server.js:236</code> → <code>server.js:134</code> (<code>revokeSession</code> — 서버에서 끊음) → <code>app.js:235</code> (<code>showLogin</code>)
자료 조회	<code>app.js:269</code> (<code>loadAll</code>) → <code>app.js:29~33</code> (Authorization 헤더 부착) → <code>server.js:145</code> (<code>requireAuth</code>) → <code>server.js:262</code> (목록, <code>user_id</code> 조건) 또는 <code>server.js:303</code> (<code>findOwnEntry</code>) → 401이면 <code>app.js:255</code> (<code>guard</code>)가 로그인 화면으로

거절을 만들어 내는 소스 — 핵심 세 곳

```
// 1) server.js:145 - 로그인 안 했으면 여기서 끝난다
function requireAuth(req, res, next) {
  const token = header.startsWith('Bearer ') ? header.slice(7).trim() : '';
  if (!token) return res.status(401)...
  const row = db.prepare('SELECT * FROM sessions WHERE token_hash = ?').get(hashToken(token));
  if (!row || row.revoked_at) return res.status(401)... // 로그아웃/비번변경으로 끊긴 토큰
  if (new Date(row.expires_at).getTime() <= Date.now()) return res.status(401)... // 만료
  req.user = user; // 주소·헤더·본문에 뭐라고 적혀 있든, 계정은 여기서만 정해진다
}

// 2) server.js:262 - 목록에도 반드시 주인 조건이 붙는다
db.prepare('SELECT * FROM entries WHERE user_id = ? ORDER BY date').all(req.user.id);

// 3) server.js:303 - 한 건을 다룰 때. 주인이 아니면 "없는 것"이 된다
function findOwnEntry(userId, id) {
  return db.prepare('SELECT * FROM entries WHERE id = ? AND user_id = ?').get(Number(id), userId) || null;
}
```

세 번째가 읽기·수정·삭제 세 경로 모두에서 가장 먼저 호출됩니다 (`server.js:309 / 315 / 322`). 특히 수정은 주인 확인을 통과한 뒤에야 저장 함수를 부르기 때문에, 거절된 요청은 데이터베이스를 한 글자도 건드리지 않습니다.

화면에서 막은 것이 아닙니다. 화면은 일기 영역을 `hidden` 으로 시작하고 토큰이 있어도 서버에 `GET /api/auth/me` 를 먼저 물어보지만, 이것은 편의일 뿐입니다. 개발자 도구로 `appView.hidden = false` 를 강제로 풀어도 화면은 비어 있습니다 — 데이터가 서버에서 오지 않기 때문입니다.

안 열리는 것을 확인한 기록

"막았습니다"라는 문장은 근거가 아니라고 해서, 실제로 요청을 보내 받은 응답을 그대로 옮겼습니다. 아래는 요약이고, 전체 요청-응답은 저장소의 [증거/](#) 폴더에 세 개 파일로 있습니다. 비밀번호는 `*****`, 토큰은 앞 8글자만 남기고 `...생략`으로 가렸습니다.

증거 파일	무엇을 확인했나
증거/확인 기록.md	1차 — 계정 A-B-C로 로컬 서버에서 전 항목 확인(628줄)
증거/테스트 계정 검증 기록.md	2차 — 테스트 계정을 새로 만들어 화면까지 눌러 가며 재확인. 이 과정에서 문제를 하나 찾아 고침(5절)
증거/배포 서버 검증 기록.md	3차 — 배포된 서버에 같은 확인. 심사자가 밟는 경로(Pages 화면 → Render 서버)까지 브라우저로 확인

확인 1 — 로그인하지 않고 자료를 부르면

```
GET /api/entries (토큰 없음) → 401 {"error":"unauthorized","message":"로그인이 필요해요."}
GET /api/entries (지어낸 토큰) → 401 {"error":"unauthorized", ...}
GET /api/entries (로그인한 토큰) → 200 내 5일 기록
```

브라우저에서 `index.html#/diary`를 직접 쳐도 화면이 서버에 물어보고 위 401을 받아 **로그인 화면**을 보여줍니다.

확인 2 — 로그아웃 뒤 같은 값으로 다시 요청하면

```
GET /api/entries Authorization: Bearer tu3WrPOR...생략 → 200 5일 기록
POST /api/auth/logout (같은 토큰) → 200 {"loggedOut":true}
GET /api/entries Authorization: Bearer tu3WrPOR...생략 → 401 {"error":"unauthorized"}
```

주소도 방식도 보낸 토큰도 같고, **달라진 것은 사이에 로그아웃을 했다는 것뿐**입니다. 브라우저에서만 지운 것이 아니라 서버의 `sessions` 행에 `revoked_at`이 찍힙니다. 비밀번호를 바꿨을 때도 같습니다(변경 전 200 → 변경 후 같은 요청이 401).

확인 3 — 남의 자료 읽기·수정·삭제 (양방향)

방향	요청	응답
A → B 읽기	<code>GET /api/entries/6</code> (A 토큰)	404 그런 기록이 없어요.
A → B 수정	<code>PUT /api/entries/6</code> (A 토큰)	404
A → B 삭제	<code>DELETE /api/entries/6</code> (A 토큰)	404
주인이 부르면	<code>GET /api/entries/6</code> (B 토큰)	200 B의 기록

B → A 읽기 / 수정 / 삭제	GET·PUT·DELETE /api/entries/1 (B 토큰)	셋 다 404
-----------------------	---	---------

403이 아니라 404를 쓴 이유: 403은 "있긴 한데 너는 안 돼"라고 알려주는 셈이라 남의 기록 id가 실제로 있다는 사실이 새어 나갑니다. 404는 그것조차 알려주지 않습니다.

거절 앞뒤로 반대편이 그대로인지: A가 B를 건드리기 전 B는 2건, 건드린 뒤에도 2건이었고 응답 JSON을 통째로 비교해도 한 글자도 다르지 않았습니다. 반대 방향도 5건 → 5건, 내용 동일. 새로 생긴 기록도 없습니다.

확인 4 — 주소·헤더·본문에 남의 계정을 적어 보내면

GET /api/entries?userId=2	(A 토큰) → 200 돌아온 id [1,2,3,4,5] = A 것뿐
GET /api/entries X-User-Id: 2	(A 토큰) → 200 돌아온 id [1,2,3,4,5] = A 것뿐
PUT /api/entries/date/2026-09-06	본문 {"userId":2,"user_id":2,...} → 200 A의 2026-09-06 으로 저장. B 목록에는 그 날짜 없음

서버는 주소·헤더·본문의 계정 번호를 읽는 코드 자체가 없습니다. 계정은 requireAuth 가 토큰에서 정한 req.user.id 하나뿐입니다.

확인 5 — 목록 격리와 비밀번호 보관

같은 주소·같은 방식(GET /api/entries)에 토큰만 바꿔 불렀습니다. A는 [1,2,3,4,5], B는 [6,7] — 겹치는 id 0개.

저장된 비밀번호는 users 테이블을 그대로 열어봤습니다. 두 계정은 똑같은 비밀번호로 만들었습니다.

id=1 email=mochi-a@example.com password_hash=\$2b\$12\$TaYwQBoF5TXHLWxJHj.2HONopQcwQ6okhZ2XRnudmbjQfbxKBM18.
id=2 email=mochi-b@example.com password_hash=\$2b\$12\$WUjpgjt2mZMN5e6wBRVZjOX7x.7xEF2TUnZ7AcwZhP5IsjUWseV0.

- 입력한 글자는 어디에도 보이지 않습니다.
- 같은 비밀번호인데 저장값이 서로 다릅니다 — bcrypt가 계정마다 다른 소금값을 붙이기 때문(\$2b\$12\$ 뒤 22 글자).
- 서버 로그 전체를 뒤져도 비밀번호 원문은 한 건도 없습니다. 로그에는 시각·방식·경로·상태 코드만 남깁니다 (server.js:184).

결들여 확인한 것	결과
같은 이메일로 두 번 가입	409 email_taken (email_lower UNIQUE)
아이디 맞고 비번만 틀림 / 아이디 자체가 없음	둘 다 401 + "이메일 또는 비밀번호가 올바르지 않아요." (문자열까지 동일)
토큰 만료	발급 12시간 뒤 401 session_expired. sessions.expires_at 에 기록
토큰이 주소에 실리는가	아니요. 항상 Authorization 헤더. 주소에 실어 보내면 무시됨
비밀키가 브라우저 코드·배포 파일·Git에 있는가	아니요. Render가 generateValue 로 만들어 서비스 환경변수에만 넣음. 나도 그 값을 본 적이 없음

계정 삭제	계정 삭제 후 사용자 행 0개, 주인 없는 일기 행 0개 (CASCADE)
-------	---

검증하면서 찾아서 고친 것 2건

둘 다 "될 것"이라고 생각하고 넘어갔다가, 실제로 눌러보고 배포해보고 나서 드러난 것입니다.

① 로그아웃했는데 앞사람 기록이 화면(DOM)에 남아 있었다

항목	내용
증상	로그아웃하면 일기 화면이 <code>hidden</code> 으로 가려지긴 하는데, 이전에 그려 둔 내용이 숨겨진 채 그대로 남아 있었음. 로그아웃 직후 숨은 영역을 읽어보니 달력 점 5개, 지표 요약 "공부 합계 285분", 일기 본문이 그대로 있었음
왜 문제인가	서버는 이미 제대로 막혀 있었지만(토큰이 끊겨 새 자료는 못 가져옴), 공용 컴퓨터에서 로그아웃한 뒤 다음 사람이 개발자 도구를 열면 앞사람이 방금 보던 일기 글이 읽혔음 . 과제 문서의 "화면에서 안 보이게 하는 것은 막은 것이 아니다"에 정확히 걸리는 경우
원인	<code>app.js</code> 의 <code>showLogin()</code> 이 상태 변수만 비우고 화면을 다시 그리지 않았음
수정	상태를 비운 뒤 <code>renderAll()</code> 을 불러 빈 상태로 다시 그리도록 함. 계정 표시·일기 입력칸도 함께 비움
확인	고친 뒤 같은 방법으로 재확인 — 숨은 달력 점 0개 , 지표 요약·규칙 변경·계정 표시·입력칸 모두 빈 상태. 배포본에서도 동일하게 확인

② 첫 배포가 실패했다 — 내가 박아둔 Node 버전 때문

항목	내용
증상	Render Blueprint 첫 배포가 Failed deploy 로 끝남
원인	<code>render.yaml</code> 에 <code>NODE_VERSION: "22.11.0"</code> 을 넣었는데, <code>node:sqlite</code> 는 22.13.0 이전에는 <code>--experimental-sqlite</code> 플래그가 있어야 쓸 수 있음 . 그래서 <code>require('node:sqlite')</code> 가 <code>ERR_UNKNOWN_BUILTIN_MODULE</code> 로 바로 죽어 서버가 뜨지 못했음. 내 로컬은 Node 24라 이 문제가 드러나지 않았음
확인	버전별로 직접 돌려봄 — 22.11.0 실패 / 22.13.0 · 22.20.0 · 24.19.0 성공
수정	<code>NODE_VERSION</code> 을 22.20.0으로 올리고, <code>package.json</code> 의 <code>engines.node</code> 도 실제 최소값 <code>>=22.13.0</code> 으로 정정. 같은 실수가 반복되지 않게 <code>render.yaml</code> · <code>server.js</code> · README에 이유를 적음
확인	22.20.0으로 서버를 실제로 띄워 health 200, 가입 201, 로그인 목록 200, 기록 저장 200, 로그아웃 뒤 401까지 확인한 뒤 재배포

잠긴 앱으로 5일 — 3일차 전에 규칙 하나를 바꿔 봄

1일차에 한 번만 정하고 끝까지 바꾸지 않은 것

항목	정한 값
답하려는 질문	공부를 아침에 시작하면 하루 공부 시간이 늘어날까?
관찰 지표 / 단위	공부 / 분
계산 규칙	그날 기록 중 공부 항목의 값을 모두 더해 하루 값으로 본다
1일차의 계획 규칙	저녁을 먹은 뒤에 공부를 시작한다

값을 다룰 때의 규칙

상황	어떻게 하나
값이 빠졌을 때	0으로 채우지 않고 "없음"으로 두며 평균의 분모에서도 뺀다. 0으로 채우면 "안 한 날"과 "적는 걸 잊은 날"이 구분되지 않기 때문
값이 중복될 때	같은 날 같은 항목이 여러 줄이면 지우지 않고 모두 더한다 (오전·오후로 나눠 적은 것으로 봄)
값이 유난히 될 때	빼지 않고 그대로 쓴다. 대신 표의 "비고"에 !를 붙여 눈에만 띄게 함. 5일짜리에서 튀는 값을 빼면 남는 게 거의 없음
반올림	화면의 합계·평균은 소수 셋째 자리에서 반올림(둘째 자리까지 표시). 저장되는 원본 값은 반올림하지 않음
주 시작 요일	월요일 (월~일)

5일 기록 (Asia/Seoul 기준)

회차	날짜	요일	공부(분)	그날의 기록
1일차	2026-09-01	화	40	저녁 먹고 시작하려다 계속 미뤄서 밤 10시에 겨우 앉았다.
2일차	2026-09-02	수	25	또 저녁 뒤로 미뤘고 졸려서 금방 접었다. 이를 다 목표 60분에 못 미쳤다.
↓ 2026-09-02 22:10 (KST) — 여기서 계획 규칙을 하나만 바꿈				
3일차	2026-09-03	목	75	아침에 일어나자마자 앉았더니 오히려 오래 갔다.
4일차	2026-09-04	금	90	아침에 바로 시작. 오늘이 제일 길었다.
5일차	2026-09-05	토	55	아침에 약속이 있어 늦게 시작했지만 그래도 앞 이틀보다는 길었다.

계획 규칙 변경 (2일차 기록 뒤 · 3일차 기록 앞)

항목	내용
바꾼 시각	2026-09-02 22:10 (KST) — 저장값 2026-09-02T13:10:00.000Z
바꾸기 전 규칙	저녁을 먹은 뒤에 공부를 시작한다
바꾼 뒤 규칙	아침에 일어나자마자 공부를 시작한다
바꾼 이유	1일차·2일차 모두 저녁으로 미루다가 목표 60분을 못 채웠다. 시작 시각만 바뀌서 하루 공부 시간이 달라지는지 보려고
무엇을 보고 바꿨나	1일차(2026-09-01) 40분, 2일차(2026-09-02) 25분 기록

바꾼 것은 **계획 규칙 하나**뿐입니다. 질문·지표·단위·계산 규칙은 그대로 두었습니다. 이 기록은 앱 안의 **계획 규칙 변경 기록**(rule_changes 표)에 남아 있습니다.

규칙 변경 전후 비교 — 같은 지표 · 같은 단위 · 같은 계산 규칙

	날 수	값	평균
바꾸기 전 (1~2일차)	2일	40, 25	32.5분
바꾼 뒤 (3~5일차)	3일	75, 90, 55	73.33분
차이			+40.83분

늘긴 했지만 결론을 내리기엔 부족합니다. 5일은 너무 짧고, 3~5일차가 목·금·토라 요일 자체의 영향(주말에 시간이 많다)과 섞여 있습니다. 5일차가 55분으로 떨어진 것도 "아침에 약속이 있었다"는 그날 사정이지 규칙 때문이 아닙니다. 그래서 이 비교는 "아침 시작이 나쁘지 않아 보인다" 정도까지만 말할 수 있습니다.

화면에 보이는 합계·평균을 손으로 더해 맞춰 보기

화면 표시: 공부 합계 285분 · 기록한 5일 평균 57분
규칙 변경 전 2일 평균 32.5분 → 변경 후 3일 평균 73.33분 (+40.83분)

손 계산: 40 + 25 + 75 + 90 + 55 = 285 → 화면 285분 같음
285 / 5 = 57 → 화면 57분 같음
(40 + 25) / 2 = 32.5 → 화면 32.5분 같음
(75 + 90 + 55) / 3 = 73.333... → 화면 73.33분 같음 (셋째 자리 반올림)

내보내기와 계정 삭제

- **내보내기** — GET /api/export 로 설정·규칙 변경·일기 전부가 **JSON 파일 하나**로 내려받아집니다. 실제 내려받은 파일을 증거/내보낸 파일 예시.json 에 넣어 두었습니다. **비밀번호 해시와 토큰은 들어 있지 않습니다.**
- **계정 삭제** — 누르면 "계정을 지우면 이 계정의 일기·습관 기록·규칙 변경 기록이 함께 영구 삭제돼요. 되돌릴 수 없어요."라고 먼저 알려주고, 두 번 확인한 뒤 지웁니다. 실제로 자료도 함께 지워지는지 일회용 계정으로 확인했습니다.

아직 못 막은 것

다 막았다고 적는 것보다 못 막은 것을 적는 쪽이 정직하다고 해서, 감추지 않고 그대로 적습니다.

못 막은 것	왜 위험한가 / 지금 상태
① 무차별 대입 (가장 큰 구멍)	로그인 요청 횟수에 아무 제한이 없어 비밀번호를 계속 넣어볼 수 있음. bcrypt 12회가 속도를 크게 늦추긴 하지만 짧거나 흔한 비밀번호면 결국 뚫림. 계정 잠금도 없어 공격이 시작돼도 본인이 알 방법이 없음. 시도 횟수 제한·실패 알림을 넣어야 하는데 시간이 모자랐음
② 비밀번호 재설정 없음	잊으면 되찾을 방법이 없음(로그인 상태에서 바꾸는 것만 됨). 급히 붙이다 "이메일만 맞으면 바뀌 준다" 같은 위험한 방식이 되기 쉬운 지점이라 아예 없다고 적어 둠
③ 토큰을 localStorage에 둠	XSS가 생기면 그 스크립트가 토큰을 읽어 감. 훔치면 비밀번호를 몰라도 남의 일기를 다 볼 수 있고 만료가 12시간이라 그만큼 피해 창이 열림. 값을 넣을 때 <code>innerHTML</code> 대신 <code>textContent</code> 를 쓰도록 정리했지만 이건 예방이지 방어가 아님. <code>HttpOnly</code> 쿠키를 쓰려면 화면·서버 주소가 달라 크로스 사이트 쿠키 문제에 걸림
④ 2단계 인증 없음	비밀번호 하나만 맞으면 들어옴. 새 나가면 그걸로 끝
⑤ 접속 기록(감사 로그) 없음	누가 로그인에 실패·성공했는지 남지 않음. 누가 내 계정을 두드렸는지, 언제 뚫렸는지 나중에 알 수 없음
⑥ 이메일 확인 절차 없음	남의 이메일 주소로도 가입 가능. 지금은 메일을 보내지 않아 피해가 없지만, 비밀번호 재설정을 붙이면 곧바로 문제가 됨
⑦ 무료 배포의 저장 한계 — 실제로 겪음	Render 무료 플랜은 디스크가 영구 저장이 아니라, 15분 넘게 요청이 없어 잠들었다 깨어나면 계정과 일기가 통째로 사라짐 . 추측이 아니라 재봤음 — 계정과 5일 기록을 넣고 20분간 요청을 전혀 보내지 않은 뒤 로그인하니 <code>401 invalid_credentials</code> . 재배포도 코드 변경도 없었음. 유료 디스크나 관리형 DB(Neon-Supabase 등)로 옮겨야 하는데 기한 안에 못 함

⑦은 인증이 뚫린 것이 아니라 호스팅의 저장 한계입니다. 가입·로그인은 언제나 되므로, 심사하시는 분이 직접 계정을 만들어 확인하시는 데에는 지장이 없습니다. 제 5일 기록은 이 보고서와 증거 파일에 그대로 남아 있습니다.

AI와 나

구분	내용
AI에게 맡긴 일	인증 백엔드(<code>server.js</code>) 전체 코드 — 계정·세션·일기 표 설계, bcryptjs 적용, 세션 발급·취소·만료, 로그인 확인(<code>requireAuth</code> , <code>findOwnEntry</code>). 6번의 <code>script.js</code> 를 서버 호출 방식으로 옮긴 <code>app.js</code> , 로그인 화면 마크업·CSS. 그리고 확인 기록을 손으로 옮겨 적는 대신 실제로 요청을 보내 응답을 그대로 받아 적고 비밀번호를 가려 주는 스크립트 — 제출한 요청·응답이 지어낸 것이 아니라 진짜 서버가 답한 것이 되도록 하기 위해서
내가 직접 판단한 일	① 인증 서비스를 쓰지 않고 직접 붙이기로 한 것(어디를 고쳤는지 보여야 하므로) ② 남의 자료 요청을 403이 아니라 404 로 돌려주기로 한 것 ③ 6번 백엔드를 고치지 않고 새 폴더로 분리해 6번 제출물을 살려 둔 것 ④ 5일 시험의 질문·지표와, 3일차 전에 바꿀 것을 "시작 시각 하나"로 좁힌 것 ⑤ 못 막은 것 7가지를 감추지 않고 그대로 적기로 한 것
AI 제안을 따르지 않은 일	AI는 처음에 JWT(<code>jsonwebtoken</code>)로 붙이자고 했습니다. 코드가 짧고 서버에 세션 표가 필요 없다는 이유였습니다. 따르지 않았습니다. JWT는 서명만 맞으면 통과시켜 로그아웃해도 만료 전까지 그 토큰이 계속 통하는데 , 이 과제의 통과 기준은 "로그아웃 뒤 같은 값으로 요청하면 거절된다"입니다. 그걸 만들려면 어차피 서버에 취소 목록을 뒤야 하니, 처음부터 서버 세션이 더 단순하고 정직하다고 판단해 방식을 바꾸게 했습니다

AI가 만든 것을 그대로 믿지 않으려고 검증을 세 번 나눠서 했습니다. 1차(로컬), 2차(테스트 계정을 새로 만들어 화면까지), 3차(배포본). 2차에서 로그아웃 뒤 DOM에 기록이 남는 문제를, 3차 직전에 배포 실패 원인을 찾았습니다(5절). 한 번만 확인하고 넘어갔다면 둘 다 놓쳤을 것입니다.

부록 A

화면 증거

모찌의 일기장

내 일기는 로그인한 뒤에만 보여요.

로그인 가입하기

이메일

mochi@example.com

비밀번호

8자 이상

로그인

▶ 서버 주소 바꾸기 (보통 건드릴 필요 없어요)

결과물 주소를 열었을 때의 첫 화면, 주소 뒤에 `#/diary` 를 직접 붙여 일기 화면으로 바로 들어가려 해도 이 로그인 화면이 나옵니다. 비밀번호 칸은 `type="password"` 라 입력해도 화면에 글자가 보이지 않습니다.

내 계정의 기록만 보임

내보내기

비밀번호 변경

계정 삭제



모찌와 함께한 지

D+8

2026.9.1부터

📖 이번 5일의 실험 설정

펼치기

질문 공부를 아침에 시작하면 하루 공부 시간이 늘어날까?

지표 공부

단위 분

계산 규칙 그날 기록 중 '공부' 항목의 값을 모두 더해 하루 값으로 본다

지금의 계획 규칙 아침에 일어나자마자 공부를 시작한다

2026년 9월						
일	월	화	수	목	금	토
		1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30			

9월 8일 화요일

오늘



최고



좋음



그냥



슬픔



화남

오늘 하루는 어땠나요? 모찌에게 이야기해주세요.

📅 오늘의 습관 기록

항목 (예: 공부)

값

단위 (예: 분)

추가

삭제

저장

📊 지표 요약

2026-09-01 ~ 2026-09-05 · 5일

공부 합계 285분 · 기록한 5일 평균 57분

규칙 변경 전 2일 평균 32.5분 → 변경 후 3일 평균 73.33분 (+40.83분)

날짜	요일	값	비고
1일자 2026-09-01	화	40분	
2일자 2026-09-02	수	25분	
3일자 2026-09-03	목	75분	
4일자 2026-09-04	금	90분	
5일자 2026-09-05	토	55분	
합계 / 평균		285분	57분

📅 이번 주 요약

9/7(월) ~ 9/13(일)

이번 주 습관 기록이 아직 없어요.

📅 계획 규칙 변경 기록

규칙 바꾸기

로그인한 뒤 보이는 내 기록. 오른쪽 위에 로그인한 계정이 표시되고, 달력에 5일치 점이 찍혀 있습니다. 실험 설정(질문·지표·단위·계산 규칙·계획 규칙), 지표 요약(합계 285분 · 평균 57분, 규칙 변경 전후 비교), 계획 규칙 변경 기록이 한 화면에 보입니다. 이 화면은 로그인하지 않으면 만들어지지 않습니다.

제출 주소와 공개 범위

항목	내용
결과물	<code>chacha1650a.github.io/assignment-repository/7번째 과제(로그인 기능)/index.html</code> 계정 없이 새 시크릿 창에서 열리며, 첫 화면은 로그인 화면
소스	<code>github.com/chacha1650a/assignment-repository</code> — 화면은 <code>7번째 과제(로그인 기능)/</code> , 서버는 <code>mochi-diary-auth-backend/</code>
인증 서버	<code>mochi-diary-auth-backend.onrender.com</code> (Render 무료, <code>render.yaml</code> Blueprint 배포). 화면에 기본값으로 들어 있어 심사자가 입력할 것이 없음
이어받은 6번	6번 최종 commit <code>f8d15118a03925b8a1e9a7d1b9574263bb65a733</code> (2026-09-04). 같은 저장소·같은 브랜치라 7번 커밋의 조상으로 포함됨
공개 범위	로그인 화면까지는 누구나. 기록은 로그인한 본인만. 심사자는 직접 가입해 확인하며, 그 계정에는 내 기록이 한 건도 보이지 않는 것이 정상
저장 위치	Render에 배포한 자체 백엔드의 SQLite. 계정별로 <code>user_id</code> 가 붙어 분리됨. 로컬 모드는 없앴(브라우저가 아니라 서버가 소유권을 책임져야 하므로)
비밀값 취급	제출물 어디에도 비밀번호·토큰·비밀키 원문 없음. 비밀번호는 <code>*****</code> , 토큰은 앞 8글자만 남기고 <code>...생략</code> . <code>SESSION_SECRET</code> 은 Render가 생성해 서비스 환경변수에만 존재하며 <code>.env</code> 는 <code>.gitignore</code> 처리

심사하시는 분께. 제 계정으로 로그인하실 필요가 없습니다. 아무 이메일로 새로 가입하시면 되고, 그 계정에 제 기록이 하나도 보이지 않는 것이 이 과제의 요점입니다. 무료 서버라 15분 이상 아무도 쓰지 않으면 잠들어서 **첫 로그인 요청만 30~60초 걸릴 수 있습니다.** 자세한 절차는 `검증 안내서.md` 에 4줄로 정리해 두었습니다.